

CS 2300 – Homework 5

Submission is done electronically via Canvas

Total possible points: 100 points

Homework Guidelines

No Handwritten Answers

This homework will require some written responses and some diagrams. For the written responses, use a standard word processor like Microsoft Word, LibreOffice Writer, Google Docs, or LaTeX. Drawings can be made using Microsoft PowerPoint or Google Drawings¹, among others (e.g. draw.io, lucidchart). You can explore other tools as well, just search around and verify the tool you are considering is compliant with the design aspects that are presented in class.

Please create your answers in a digital format, export to a PDF, and upload that PDF to Canvas. No need to turn a printed copy. For those of you considering LaTeX, ShareLaTeX and Overleaf are good online editors.

One Problem per Page

In the past, many students turned in a single piece of paper with all answered questions smooshed onto it. This did not leave my grader or I with adequate room to provide suggestions on how a design could be improved nor explanations on errors. The process of design work is to work in iterative stages: the first stage will be rather messy (and can be hand-written), the second stage will have things cleaned up, and so on. The final stage is what you should turn in, and it should be free of scratched-out work and high-density drawings. Only answer one problem on *at least* one page, and refrain from super-compact answers that leaves more than 1/2 of the page empty. Ideally, leave somewhere between 1/4 to 1/2 of a page empty so that we can make annotations.

¹ <https://docs.google.com/drawings/>

Environment Setup Instructions (Required for All Students)

To ensure everyone works in a consistent and stable SQL environment and reduce platform differences, in this assignment, we will be using Docker and Docker Compose to launch a pre-configured PostgreSQL database along with pgAdmin, a web-based GUI for SQL development.

Docker is a widely adopted tool in industry that allows us to create lightweight, consistent, and portable virtualized development environments across different systems. It acts like a virtual environment manager, allowing developers to package applications with all their essential dependencies into isolated containers¹ that run reliably and smoothly on any machine.

Docker Compose is a tool for defining and running multi-container applications. It is the key to unlocking a streamlined and efficient development and deployment experience.

PostgreSQL is a powerful, open-source, relational database systems used by companies, developers, and researchers around the world. It supports standard SQL with advanced capabilities like subqueries, joins, and views. It handles large-scale, complex data with strong reliability.

Step 1: Install Docker Desktop

Docker Desktop is a GUI application that makes it easy to use Docker on most computer platforms. It includes everything you need to build, run, and manage containers on your computer.

- **macOS:** <https://docs.docker.com/desktop/setup/install/mac-install/>
 - It is strongly recommended to install Docker Desktop via *Homebrew*. This will make updating and managing Docker much (a lot!) easier in the long run, especially when newer versions or patches are released.
- **Windows:** <https://docs.docker.com/desktop/setup/install/windows-install/>
- **Linux:** <https://docs.docker.com/desktop/setup/install/linux/>

If you're unable to install Docker for any reason, please contact me early to discuss alternative setup options (e.g., installing PostgreSQL workbench locally, using *Replit* or a cloud-based PostgreSQL instance).

Step 2: Download the Project Folder

Download the provided ZIP file containing:

- *docker-compose.yml*
- *init/init.sql* (preloaded with tables and data for this assignment)

1. A Docker Container is a lightweight, standalone, and portable runtime environment that includes the application, its dependencies, and everything else it needs to run (libraries, config files, etc.). It's just like a mini virtual computer that runs just your app—but way more efficient than a full virtual machine.

Step 3: Launch (Docker Up) the Database Environment

In your computer's terminal or command prompt, navigate into the project folder and run:

docker compose up

This will:

- Start up PostgreSQL with the predefined database *cs2300db*
- Load the assignment schema and data automatically
- Launch *pgAdmin* and ready for connections

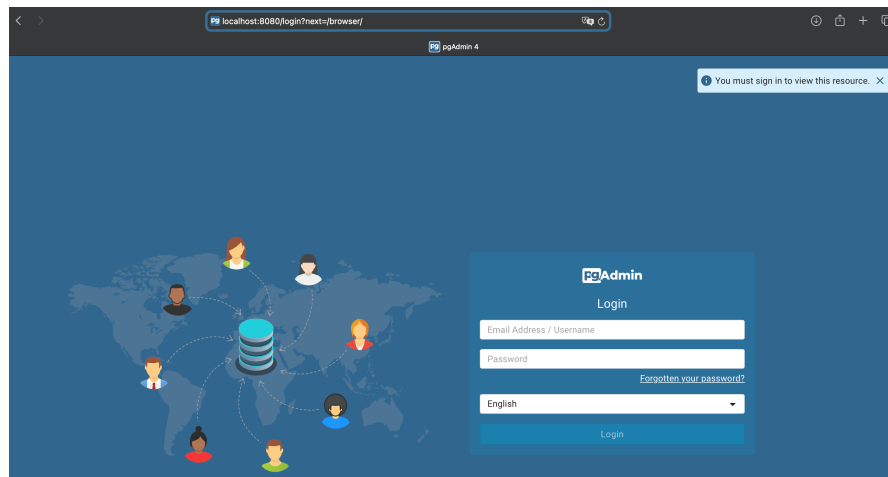
You should see output messages in the terminal showing that the services are starting successfully.

```
(+) Running 4/4
✓ Network cs2300-docker-sql_default Created 0.1s
✓ Volume "cs2300-docker-sql_pgdata" Created 0.0s
✓ Container cs2300-db Created 0.2s
✓ Container cs2300-pgadmin Created 0.2s
Attaching to cs2300-db, cs2300-pgadmin
cs2300-db The files belonging to this database system will be owned by user "postgres".
cs2300-db This user must also own the server process.
cs2300-db The database cluster will be initialized with locale "en_US.utf8".
cs2300-db The default database encoding has accordingly been set to "UTF8".
cs2300-db The default text search configuration will be set to "english".
cs2300-db Data page checksums are disabled.
cs2300-db fixing permissions on existing directory /var/lib/postgresql/data ... ok
cs2300-db creating subdirectories ... ok
cs2300-db selecting dynamic shared memory implementation ... posix
cs2300-db selecting default "max_connections" ... 100
cs2300-db selecting default "shared_buffers" ... 128MB
cs2300-db selecting default time zone ... Etc/UTC
cs2300-db creating configuration files ... ok
cs2300-pgadmin email config is {'CHECK_EMAIL_DELIVERABILITY': False, 'ALLOW_SPECIAL_EMAIL_DOMAINS': [], 'GLOBALLY_DELIVERABLE': True}
cs2300-pgadmin running bootstrap script ... ok
cs2300-db performing post-bootstrap initialization ... ok
cs2300-db initdb: warning: enabling "trust" authentication for local connections
cs2300-db initdb: hint: You can change this by editing pg_hba.conf or using the option -A, or --auth-local and --auth-host, the next time you run initdb.
cs2300-db syncing data to disk ... ok
cs2300-db Success. You can now start the database server using:
cs2300-db pg_ctl -D /var/lib/postgresql/data -l logfile start
cs2300-db waiting for server to start....2025-04-13 03:18:35.298 UTC [47] LOG: starting PostgreSQL 17.4 (Debian 17.4-1.pgdg120+2) on x86_64-pc-linux-gnu, compiled by gcc (Debian 12.2.0-14) 12.2.0, 64-bit
cs2300-db 2025-04-13 03:18:35.299 UTC [47] LOG: listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
cs2300-db 2025-04-13 03:18:35.306 UTC [50] LOG: database system was shut down at 2025-04-13 03:18:35 UTC
cs2300-db 2025-04-13 03:18:35.312 UTC [47] LOG: database system is ready to accept connections
cs2300-db done
cs2300-db server started
cs2300-db CREATE DATABASE
```

Step 4: Log Into pgAdmin

Access *pgAdmin* on your web browser of choice at <http://localhost:8080>. This will launch *pgAdmin*, where you'll log in and connect to the preloaded database.

- **Email:** student@cs2300.edu
- **Password:** *admin123*



If you encounter an error message saying that the port is already in use, it's likely something else on your computer is using port 8080. To fix this:

1. Change the *pgAdmin* port in *docker-compose.yml*
`ports:`
`- "8080:80"`
2. Replace *8080* with another available port, like *8888*, *8090*, *9090*, etc.
3. Save the file, then shut down and restart the docker containers using the terminal:
`docker compose down`
`docker compose up`

Step 5: Connect to Your Database in *pgAdmin*

- Click on "Add New Server"
- Go to the *General* tab and name the server *CS2300 PostgreSQL*

Register - Server

General

Connection

Parameters

SSH Tunnel

Advanced

Post Connection SQL

Tags

Name

CS2300 PostgreSQL

Server group

Servers

Background

X

Foreground

X

Connect now?

Shared?

Shared Username

Comments

i

?

X Close

Reset

Save

- Go to the *Connection* tab and enter:
 - **Host name:** *db*
 - **Port:** *5432*
 - **Maintenance database:** *cs2300db*
 - **Username:** *student*
 - **Password:** *pass123*

 **Register - Server** ×

General

Connection

Parameters

SSH Tunnel

Advanced

Post Connection SQL

Tags

Host name/address

db

Port

5432

Maintenance database

cs2300db

Username

student

Kerberos authentication?

☐

Password

.....

Save password?

☐

Role

Service

×

 Close

↺

 Reset

💾

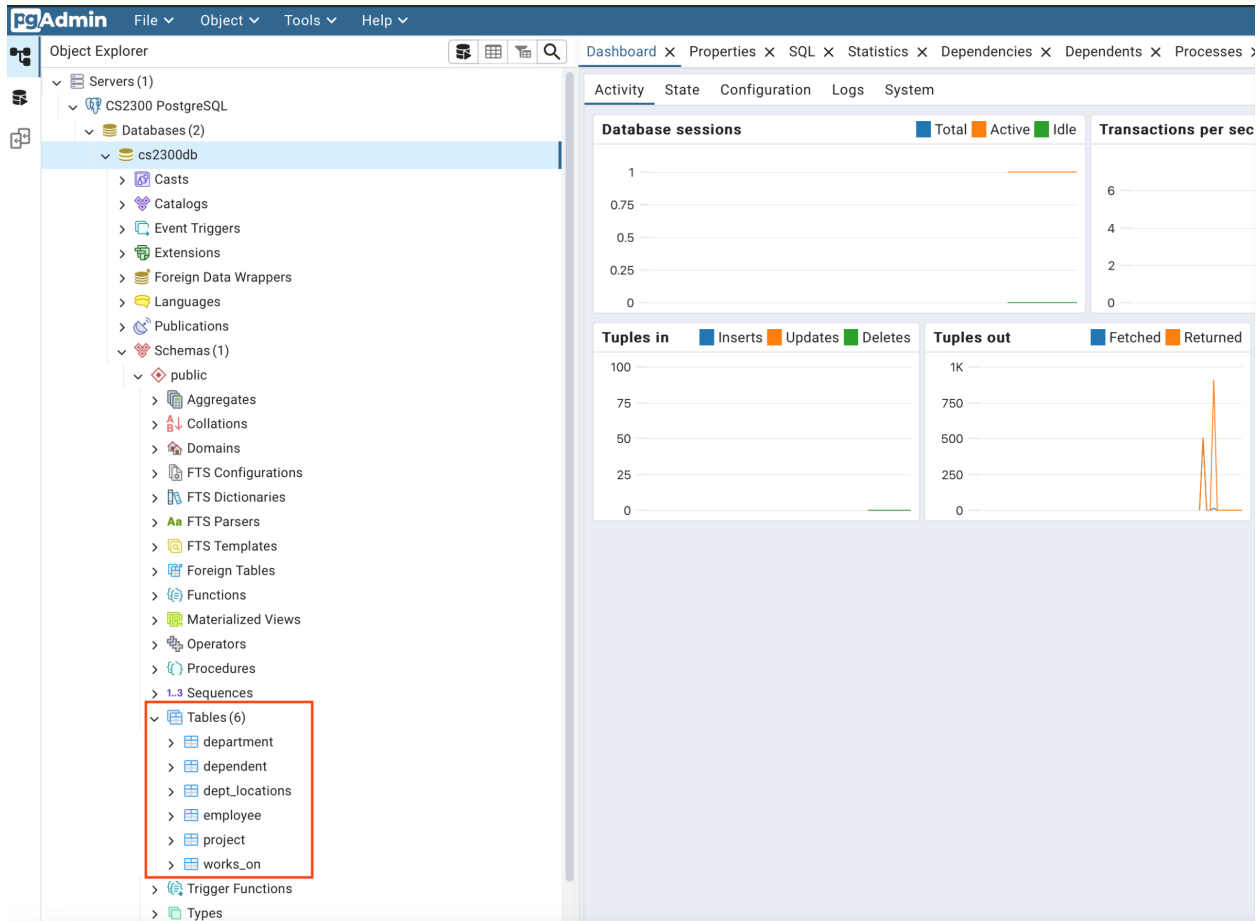
 Save

- Click *Save*
- You are now ready to begin writing and testing your SQL!

Problem 1

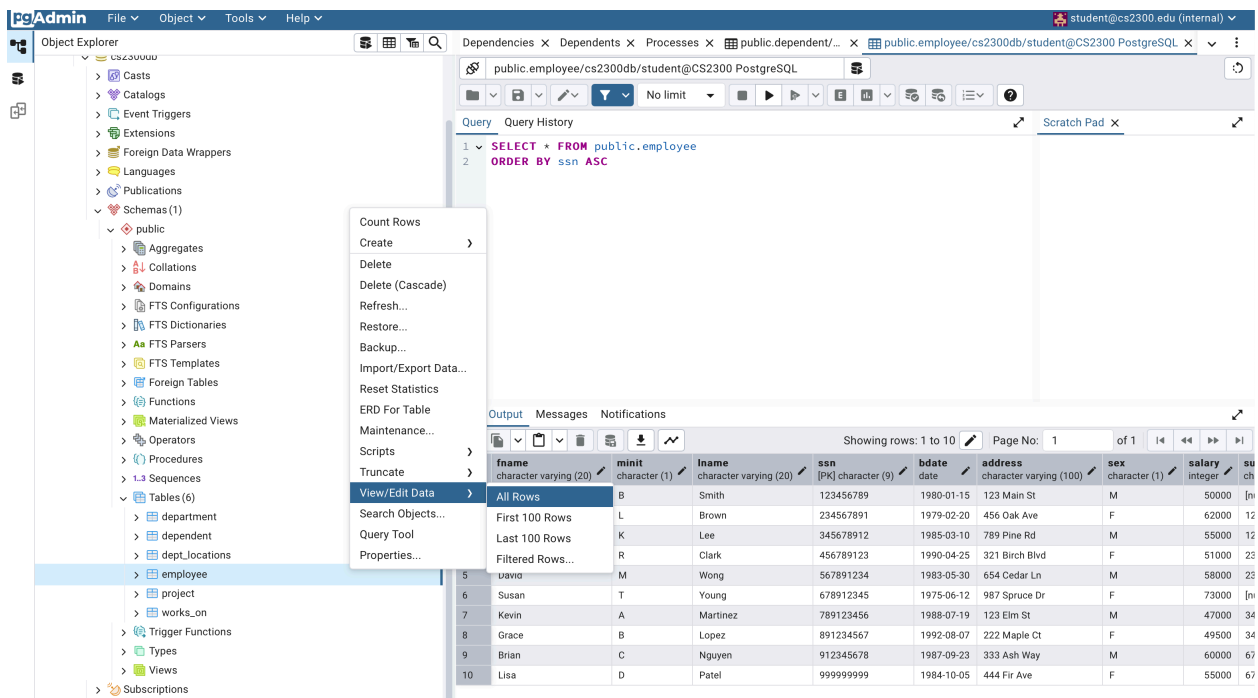
Using the preloaded database (*cs2300db*) based on the company schema shown in Figure 1, write ONE SQL file (extension .sql) that contains the SQL queries to the following data queries.

You do not need to create any tables or insert data—everything has already been created for you inside the provided database environment. Your only task is to write correct, efficient, and readable SQL queries.



To quickly preview the rows in a table in *pgAdmin*:

1. Right-click on the desired table.
2. Hover over *View/Edit Data*, then click *All Rows*.



Important: You must write standard SQL (PostgreSQL-compliant) and test your solutions in *pgAdmin* or your own SQL tool. All your queries must run without errors in *pgAdmin* or any PostgreSQL environment.

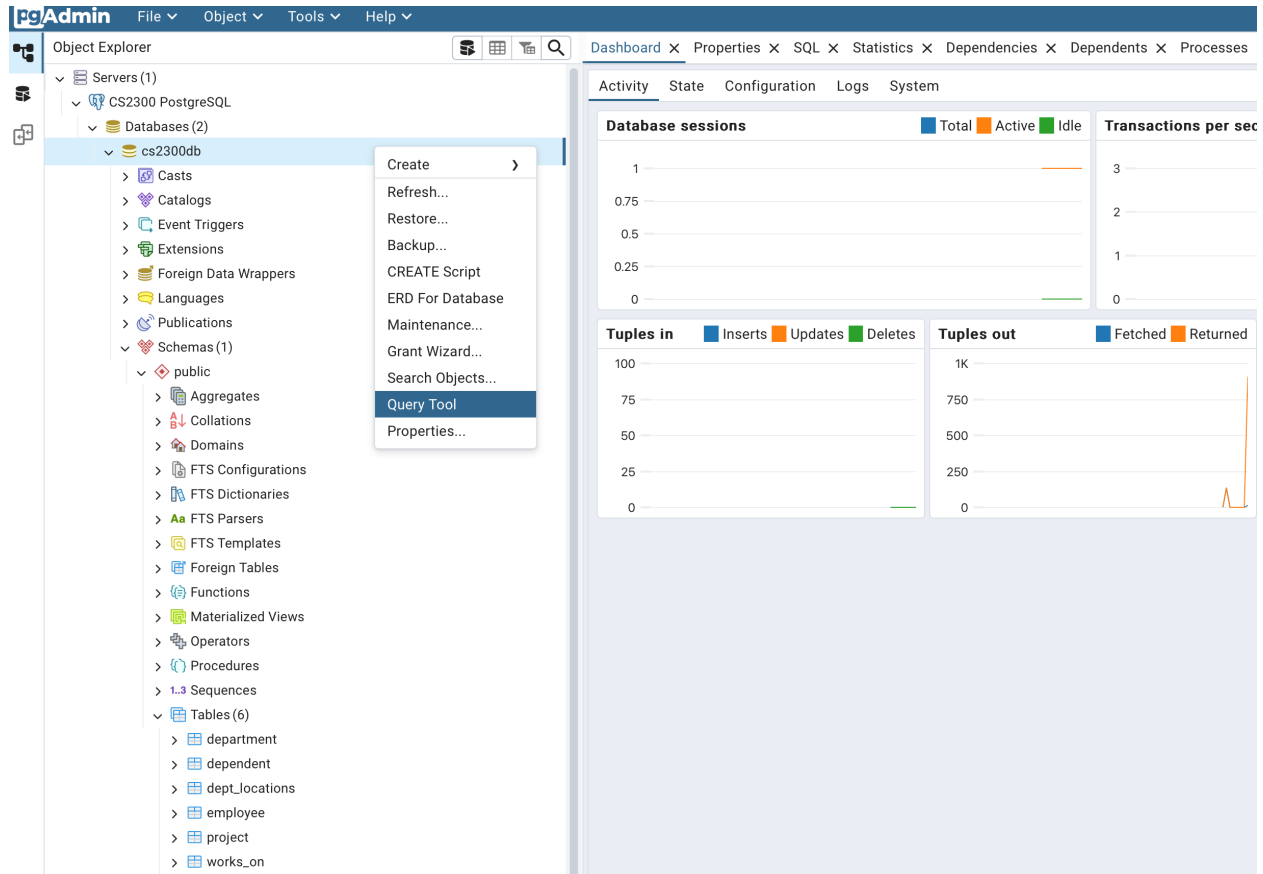
Although you are free to use any editor or environment you prefer, below are some recommended tools that work well with PostgreSQL and are easy to set up:

- **pgAdmin**
 - A full-featured PostgreSQL GUI client this assignment uses
- **DBeaver**
 - A cross-platform SQL GUI
 - Supports PostgreSQL and many other databases
- **DataGrip**
 - A professional SQL IDE by JetBrains (free with a student license)
 - Smart code completion, inline explanations, and syntax checks
- **Visual Studio Code (VS Code)**
 - Requires the “SQLTools” or “PostgreSQL” extension

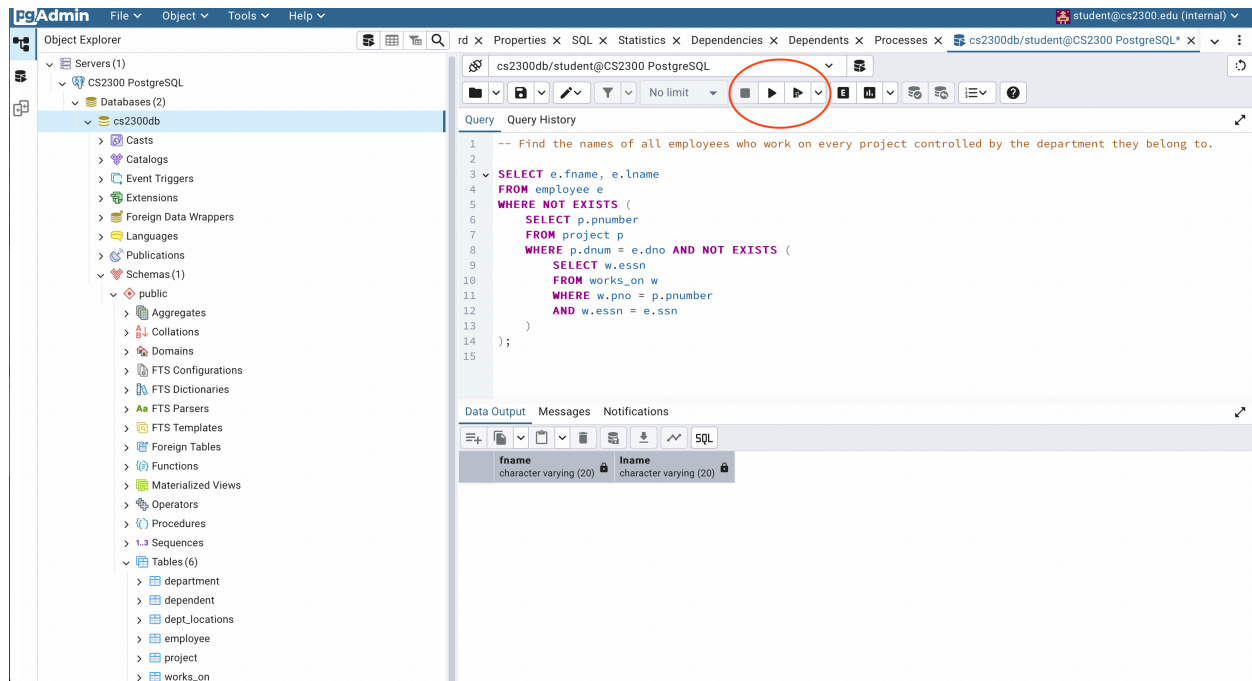
You may connect any of the above tools to the running Docker PostgreSQL container using the connection details shown previously in the Setup Instructions—Step 5.

Develop and Test SQL File in pgAdmin:

1. Right-click on the database of interest and select *Query Tool*.
2. The *Query Tool Workspace* will appear for you to start writing SQL queries in the editor.



3. After writing a query (or multiple), click on the *Execute/Play* button to show the result or error messages if the query is invalid or incorrect.



4. Make sure to save your work in a file with a .sql extension.

Data Queries:

- Retrieve the names of all employees in department 5 who work more than 10 hours per week on the ProductX project.
- List the names of all employees who have a dependent with the same first name as themselves.
- Find the names of all employees who are directly supervised by 'Franklin Wong'.
- For each project, list the project name and the total hours per week (by all employees) spent on that project.
- Retrieve the names of all employees who work on every project.
- Retrieve the names of all employees who do not work on any project.
- For each department, retrieve the department name and the average salary of all employees working in that department.
- Retrieve the average salary of all female employees

- i. Retrieve all employees by name and SSN who have cumulatively worked more than 100 hours across all projects. For each qualifying employee, also include the total hours worked and the name and manager SSN of their respective departments. Additionally, the result should account for departments that lack employees who meet this threshold.

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
-------	-------	-------	------------	-------	---------	-----	--------	-----------	-----

DEPARTMENT

Dname	<u>Dnumber</u>	Mgr_ssn	Mgr_start_date
-------	----------------	---------	----------------

DEPT_LOCATIONS

<u>Dnumber</u>	<u>Dlocation</u>
----------------	------------------

PROJECT

Pname	<u>Pnumber</u>	Plocation	Dnum
-------	----------------	-----------	------

WORKS_ON

<u>Essn</u>	<u>Pno</u>	Hours
-------------	------------	-------

DEPENDENT

<u>Essn</u>	<u>Dependent_name</u>	Sex	Bdate	Relationship
-------------	-----------------------	-----	-------	--------------

Figure 1

Problem 2

Write ONE SQL file (extension .sql) that creates a simple database, detailed below. Submit your .sql file through Canvas.

Construct your SQL statements in accordance with the assumptions stated below. Your database definition should include **all necessary table constraints** (Primary Key, Foreign Key, NOT NULL, DOMAIN, CHECK, etc.) based on the assumptions listed. Apply all relevant constraints covered in lecture. When defining foreign key constraints, don't forget to explicitly specify how the database should respond when a referenced row is deleted (**ON DELETE...**) or updated (**ON UPDATE...**) in the parent table.

AlbumTrack(<u>rid:int</u> , <u>cat#:str</u> , <u>sid:int</u> , trackno:int)	Label(<u>lid:int</u> , lname:str, labbr:str)
Song(<u>sid:int</u> , stitle:str, duration:int, remixof:int, artist:int)	Release(<u>rid:int</u> , rtitle:str, year:int, aid:int)
Rerelease(<u>catno:str</u> , <u>rid:int</u> , upc:str, label:int, year:int, medium:str)	Artist(<u>aid:int</u> , aname:str)

Figure 2: Sample Relational Model

Assumptions:

- Song durations are in units of seconds and must be > 0
- Years cannot be before 1900
- Unless otherwise stated, assume anything that is a string will not exceed 80 characters in length
- UPCs may not be longer than 12 characters
- Rerelease.medium can only be one of the following values: CD, Web, LP, 45, Tape
- Label.labbr will not be longer than 5 characters
- Any column in the relations in Figure 2 that does not have any NULL values (empty cells) should be assumed to have a NOT NULL constraint. Likewise, any column with a least one null value should be assumed to be nullable.

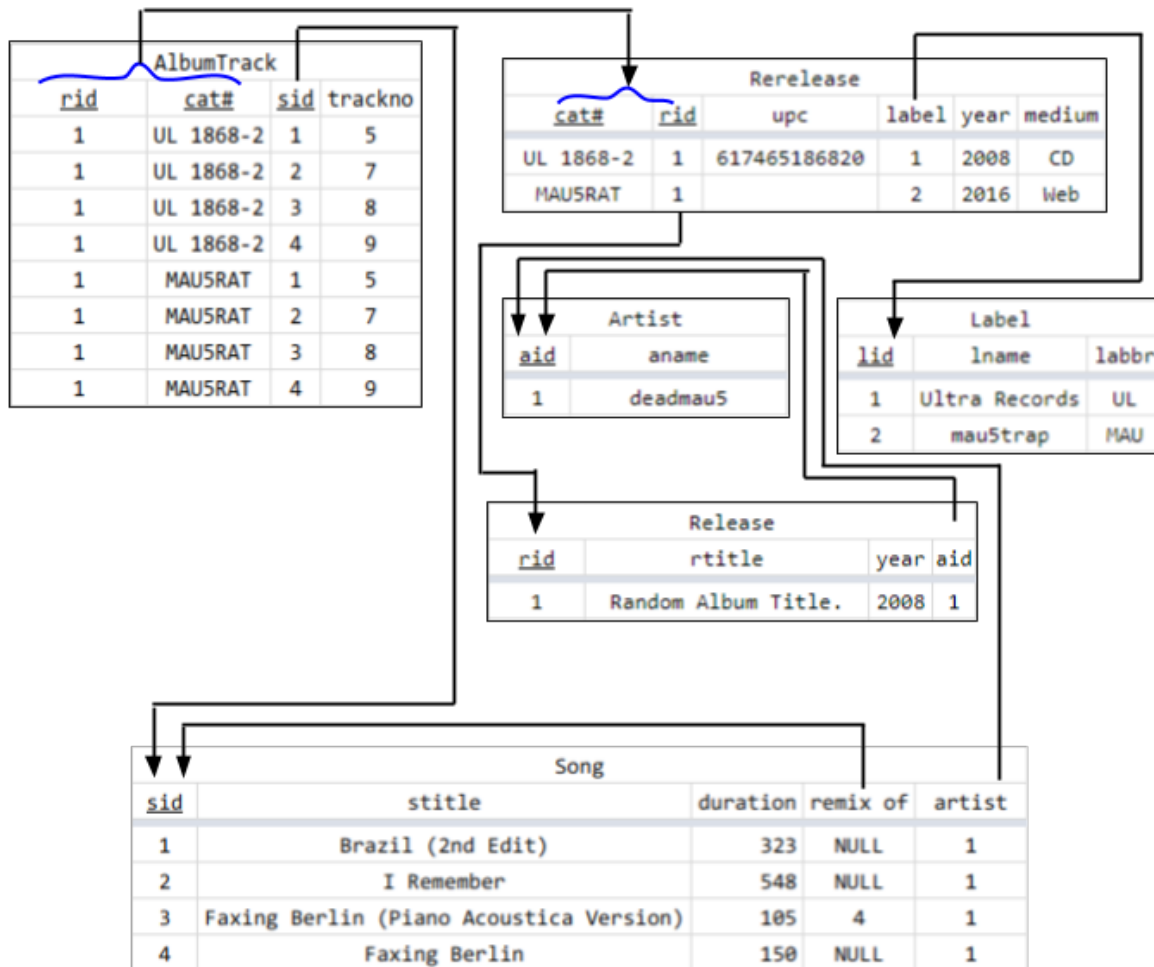


Figure 3: Sample Database State for Problem 2.

To-do:

- (1) Before you can develop your .sql file for this problem set, you must first create a new, empty PostgreSQL database in *pgAdmin*.
 - a. In the *Object Explorer*, right click on *Databases* to create a new database.

The screenshot shows the PgAdmin interface with a SQL query executed. The query is designed to find employees who work on every project controlled by their department. The results are displayed in a table with two columns: **fname** and **lname**.

```

1  -- Find the names of all employees who work on every project controlled by the department they belong to.
2
3  SELECT e.fname, e.lname
4  FROM employee e
5  WHERE EXISTS (
6      SELECT p.pnumber
7      FROM project p
8      WHERE p.dnum = e.dno AND NOT EXISTS (
9          SELECT w.essn
10         FROM works_on w
11         WHERE w.pno = p.pnumber
12             AND w.essn = e.ssn
13     )
14 );
15

```

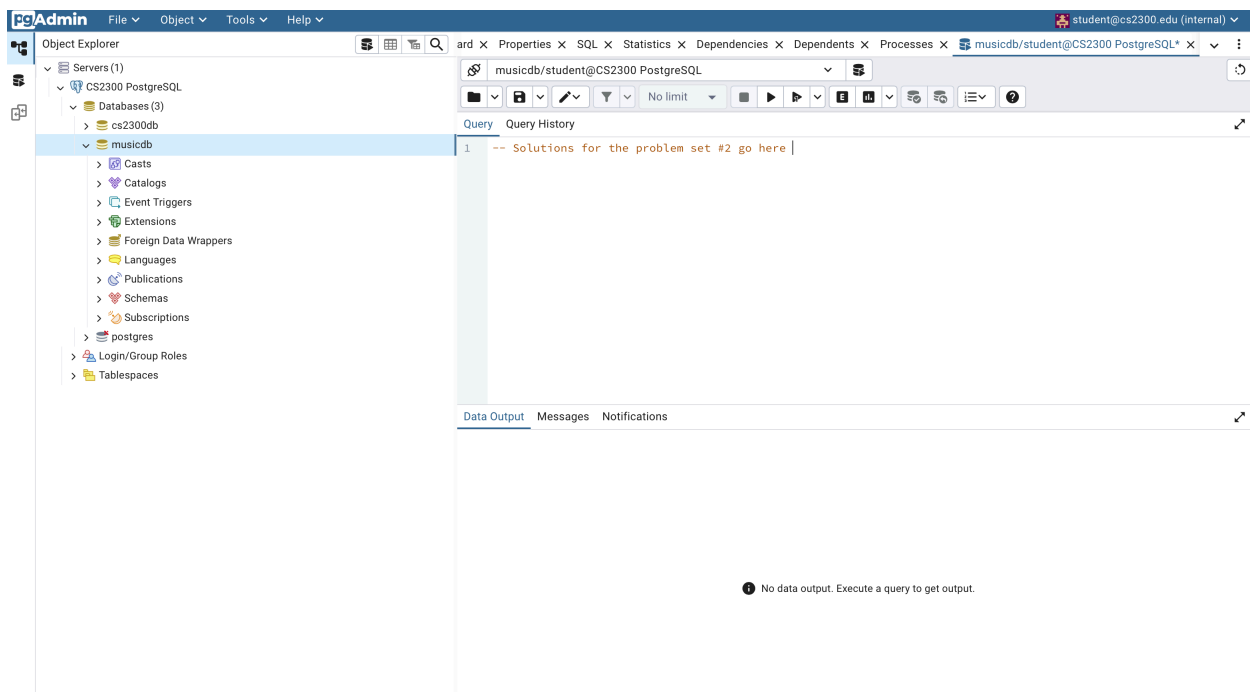
fname	lname
John	Smith
Anna	Brown
James	Lee
Emily	Clark
David	Wong
Susan	Young
Kevin	Martinez
Grace	Lopez
Brian	Nguyen
Lisa	Patel

- b. Enter a name for your database, *musicdb*, and leave all other options as default.

The screenshot shows the 'Create - Database' dialog box in PgAdmin. The 'General' tab is selected, and the 'Database' field is filled with 'musicdb'. The 'Owner' is set to 'student'. The 'Comment' field is empty. The 'Save' button is highlighted.

fname	lname
James	Lee
Emily	Clark
David	Wong
Susan	Young
Kevin	Martinez
Grace	Lopez
Brian	Nguyen
Lisa	Patel

- c. You will now see *musicdb* listed under the *Databases* section. This is where you'll run your SQL script to create tables and insert data.



(2) Create 6 tables as shown in Figure 1.

- a. You can rename attribute names appropriately - e.g. rename cat# to catno or catalog_number or whatever.

(3) Insert values into the tables as shown in Figure 3.

(4) Create SELECT queries to show the contents of each table.

(5) Implement the following SQL queries.

- a. Find the song titles from songs included on rereleases that were released both after 2008 and on the 'Ultra Records' label.

- i. *Sample Output*

stitle

- b. For each rerelease, list its associated release's title and the rerelease's year and the number of songs it contains.

- i. *Sample Output*

rtitle	year	album_track_count
Random Album Title.	2016	4
Random Album Title.	2008	4

- c. Compute the runtime (in seconds) of each rerelease and include its catalog number and associated release title.

i. *Sample Output*

rtitle	catno	total_duration
Random Album Title.	MAU5RAT	1126
Random Album Title.	UL 1868-2	1126

- d. For each remixed song, display it's title, duration, (remixing) artist name, and the title and (original) artist name who it remixed.

i. *Sample Output*

remix	duration	remixing_artist	original	original_artist
Faxing Berlin (Piano Acoustica Version)	105	deadmau5	Faxing Berlin	deadmau5

- (6) Drop the tables in an order so that no referential integrity constraints are violated (and thus no CASCADEs are necessary). Once executed, all table definitions and all its data should be completely removed from the *musicdb* database.

- a. The goal here is to drop **child tables** first, then their **parent tables**, in a way that doesn't require "*DROP TABLE ... CASCADE;*".

Notes:

- **Use DROP TABLE IF EXISTS**

When you are developing and testing your .sql script file, you'll often run the script multiple times in *pgAdmin* or another tool. If the file only contains "*CREATE TABLE*" statements like this:

```
CREATE TABLE Song(...);
```

And then try to run it again without removing the old table first, PostgreSQL will throw an error such as "*ERROR: relation "song" already exists*". To avoid this, you can **first drop the table if it already exists**, like this:

```
DROP TABLE IF EXISTS Song;
```

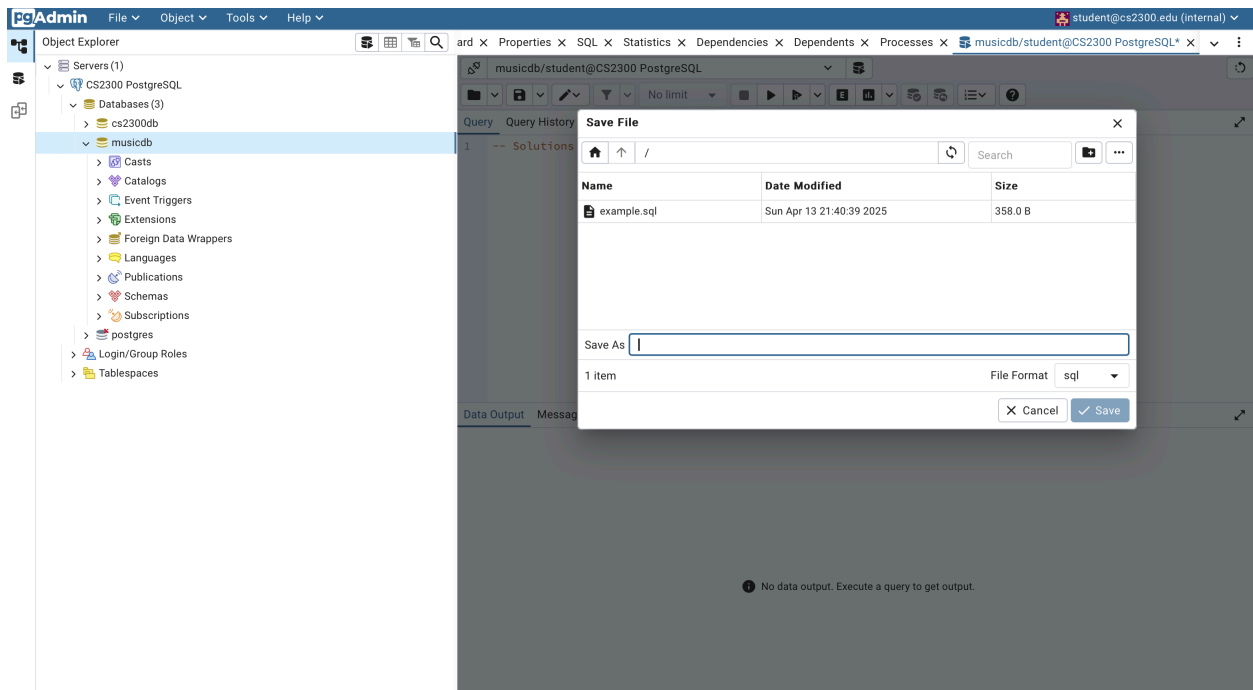
```
CREATE TABLE Song(...);
```

This approach ensures a clean slate every time you run the SQL file without errors, which is useful during development, debugging, or when schema changes.

- **Saving and Continuing Later**

At any point of completing this homework assignment, you can save your work any time in *pgAdmin*'s query editor workspace. This allows you to preserve your SQL script and continue editing later.

- Since our Docker Compose file is set up with defined storage volumes, your PostgreSQL database data is **persistent** even if you shut down your containers.
- You can safely stop your Docker container (with *docker compose down*) or simply quit Docker Desktop.
- When you start it again later (with *docker compose up*), your database and its data will still be there—no need to recreate it every time.



- Reminder, you must submit **only one .sql file** that contains:
 - All *“CREATE TABLE”* statements with appropriate constraints.
 - ALL relevant *“INSERT”* statements.
 - ALL required SQL queries.
 - ALL *“DROP TABLE”* statements.